mSEC-AT

Evaluation of mSEC-AT execution time, output consistency and expert-based validation

Quantitative evaluation of execution time, scalability trends, requirement-level output consistency, and expert-based validation

Document purpose	Quantitative evaluation report of mSEC-AT execution time, scalability trends, requirement-level output consistency and expert-based validation
Scope	70 complete executions across 7 Android applications, with 10 executions per application
Evaluation focus	Execution time, application-level size and complexity indicators, exploratory effort-estimation models, and requirement-level repeatability
Execution environment	Dedicated self-hosted GitHub Actions runner with MobSF, Android Emulator, and Ollama/gpt-oss
Prepared date	20 Jun 2026

Contents

Evaluation overview and research questions 3

1. Evaluation setup 4

2. Execution-time results 7

3. Operation-level execution time 8

4. Scalability analysis: repository size and execution time 9

5. Association between application characteristics and execution time 11

6. Requirement-level output stability and expert-based validation 14

7. Threats to validity 16

8. Data availability 17

9. Conclusions 18

References 19

Evaluation overview and research questions

The execution time and requirement-level output consistency of mSEC-AT were examined through a quantitative assessment of the complete auditing pipeline. The aim was to characterise the runtime cost of the tool, analyse how execution time varies with application size and complexity, and assess whether the structured requirement-level outputs remain consistent across repeated executions.

To make the results interpretable, execution time was analysed together with application-level size and complexity indicators, including repository size, APK size, lines of code (LOC), unadjusted function points (UFP) as a functional-size indicator [1], number of scanned files, Android components, exported components, and number of collected findings. For each application, the complete pipeline was executed ten times under the same experimental configuration.

The following evaluation questions were formulated:

RQ1. What execution time can be expected when running the complete mSEC-AT pipeline on Android applications with different size and complexity profiles?

RQ2. Are the requirement-level classifications produced by mSEC-AT consistent across repeated executions under the evaluated configuration?

1. Evaluation setup

The evaluation considered seven open-source Android applications, with ten complete executions of the mSEC-AT pipeline per application. This repeated-execution design was used to quantify runtime variability and assess the stability of requirement-level classifications under a controlled execution environment.

1.1. Subject application selection

The subject applications were selected to cover heterogeneous Android project profiles rather than to form a statistically random sample of the Android ecosystem. All selected applications are open source and provide public repositories and analysable Android artefacts, which is necessary because the complete mSEC-AT pipeline requires access to source code, dependency metadata, APK generation, and MobSF analysis.

The selection aimed to include variability in repository size, APK size, lines of code (LOC), Kotlin and Java files, total scanned files, unadjusted function points (UFP), Android components, exported Android components, and the amount of security evidence generated by the integrated tools. The set combines real-world production applications with intentionally vulnerable or security-oriented applications, allowing the pipeline to be exercised over large mature projects, medium-sized applications, smaller applications, and applications with a higher density of security-relevant evidence.

The selected applications should be interpreted as a heterogeneous benchmark set for execution-time and scalability assessment, not as a statistically representative sample of all Android applications. The goal was not to compare the security quality of the applications, but to evaluate the execution cost of mSEC-AT across different Android project profiles. The selection criteria are summarised in ***Table 1***.

Table 1. Selection criteria considered for the subject applications.

Criterion	Rationale
Open-source repository	Enables reproducibility and source-code inspection.
Buildable Android project	Required to execute the full pipeline and analyse the generated Android artefact.
Android-specific structure	Ensures that manifest, component, dependency, source-code, and APK analysis can be exercised.
Diversity in size	Supports scalability analysis using repository size, APK size, LOC, and scanned files.
Diversity in complexity	Allows execution time to be related to UFP, Android components, exported components, and findings.
Real-world and vulnerable/security-oriented apps	Covers both realistic production scenarios and cases rich in security evidence.

1.2. Execution environment

All executions were performed on the same dedicated workstation, configured as a self-hosted GitHub Actions runner and hosting the required supporting services, including MobSF, the Android Emulator, and the LLM service used during the executions. The LLM was served through Ollama using the gpt-oss model, and the same LLM backend, model, and configuration were kept unchanged across all executions. Keeping the hardware, network, and LLM execution environment unchanged helped reduce variability caused by differences in computational resources or model-serving conditions. ***Table 2*** reports the hardware and software configuration used throughout the evaluation.

Table 2. Hardware and execution environment used in the evaluation.

Component	Specification
Execution mode	Self-hosted GitHub Actions runner
Supporting services	MobSF 4.5, Android emulator and Ollama
Motherboard	ASUS ROG STRIX B660-F GAMING WIFI, LGA1700
CPU	Intel Core i7-12700, 12th Gen
Graphics	Integrated graphics
Memory	64 GB DDR5 4800 MHz
Storage	1 TB Samsung 970 EVO Plus NVMe PCIe SSD
Operating system	Microsoft Windows Server 2022
Network	Integrated Gigabit Ethernet; 1 Gbps symmetric Internet connection.
LLM backend	Ollama
LLM model	gpt-oss – 20b
LLM serving host	Intel Core Ultra 7 265KF 3.9 GHz, 128 GB DDR5 5600 MHz, NVIDIA GeForce RTX 5080 OC with 16 GB GDDR7

The reported execution times should therefore be interpreted as measurements obtained under this specific execution environment. Stages such as compilation, MobSF processing, Android emulation, dependency resolution, and external service communication may be affected by hardware and network conditions. The results are intended primarily to characterise the relative cost of the mSEC-AT pipeline stages and their relationship with application size and complexity.

The GitHub Actions workflow was parameterised through repository variables to support the different build requirements of the evaluated Android applications. Some variables were shared across all executions, whereas others were configured per application, such as the Java version and Gradle build command. Secret values, including API keys and signing material, are intentionally not reported. ***Table 3*** and ***Table 4*** summarise the shared and application-specific repository variables used during the evaluation.

Table 3. Shared GitHub Actions repository variables used during the evaluation.

Variable	Value used in the evaluation	Purpose
ENABLE_CODE_SCANNING_UPLOAD	true	Enables upload of code-scanning results when supported by the workflow.
LLM_BASE_URL	https://ai4se.inf.um.es:8443/v1	Defines the OpenAI-compatible endpoint used by the LLM calls.
LLM_MODEL	gpt-oss	Defines the LLM backend used during the evaluated executions.

Table 4. Application-specific GitHub Actions repository variables used during the evaluation.

Application	JAVA_VERSION	ANDROID_BUILD_CMD
Signal	17	.\gradlew.bat --no-daemon clean :Signal-Android:assemblePlayProdRelease
K-9 Mail	21	.\gradlew.bat --no-daemon clean :app-thunderbird:assembleRelease
AntennaPod	21	.\gradlew.bat --no-daemon clean :app:assemblePlayRelease
Markor	21	.\gradlew.bat --no-daemon clean :app:assembleFlavorGplayRelease
F-Droid	17	.\gradlew.bat --no-daemon clean assembleRelease
DVAC	21	.\gradlew.bat --no-daemon clean assembleRelease
OpenMRS	11	.\gradlew.bat --no-daemon clean assembleRelease

Note. ANDROID_BUILD_CMD defines the Gradle command executed by the workflow, and JAVA_VERSION defines the Java runtime used for the corresponding application. Shared repository variables were kept fixed across the evaluated executions. Repository secrets, such as LLM_API_KEY, MOBSF_API_KEY, and Android signing credentials, were required for execution but are not reported for security reasons.

For each execution, we measured the total execution time and the duration of the main timed operations in the pipeline, including compilation, CodeQL analysis, Trivy scanning, MobSF processing, Vision360 generation, security requirement evaluation, structured JSON generation, and final summary generation. Timed operations are reported independently because some of them are executed in parallel or are nested within broader stages of the workflow. ***Table 5*** summarises the subject applications together with the size and complexity indicators used in the execution-time analysis.

Table 5. Subject applications and size indicators used in the execution-time evaluation. "Repo MB": repository size in megabytes; "APK MB": APK size in megabytes; "LOC": lines of code; "UFP": unadjusted function points; "Scanned files": total number of source files scanned; "Android comp.": Android components; "Mean time": mean total execution time.

Application	Repo MB	APK MB	LOC	UFP	Scanned files	Android comp.	Mean time (min.)
Signal	2,360.0	240.5	1,603,006	12,500	8,737	161	29.83
K-9 Mail	248.0	10.6	129,770	1,183	7,139	18	14.74
AntennaPod	134.0	10.1	62,675	1,180	1,165	46	9.59
Markor	58.2	11.3	30,512	576	797	17	7.29
F-Droid	79.7	11.9	94,671	716	2,086	18	9.01
DVAC	37.3	10.3	2,854	79	120	18	6.78
OpenMRS	12.1	11.0	53,765	462	839	49	7.00

2. Execution-time results

At the application level, the mean total execution time ranged from 6.78 minutes for DVAC to 29.83 minutes for Signal Android. Across the 70 individual executions, the observed total execution times ranged from 6.20 minutes, corresponding to the fastest DVAC run, to 38.13 minutes, corresponding to the slowest K-9 Mail run. ***Figure 1*** shows the mean total execution time per application, and ***Table 6*** reports the corresponding descriptive statistics, including mean, standard deviation, coefficient of variation, minimum, maximum, mean audit time, and mean compilation time.

Table 6. Descriptive statistics of total execution time by application. "Mean": arithmetic mean; "SD": standard deviation; "CV": coefficient of variation, computed as SD/Mean × 100; "Min" and "Max": minimum and maximum observed values; "Mean audit": average audit-stage time; "Mean compile": average compilation time.

Application	Mean (min)	SD (min)	CV(%)	Min (min)	Max (min)	Mean audit (min)	Mean compile (min)
Signal	29.83	4.18	14.00	25.92	35.73	20.05	9.78
K-9 Mail	14.74	7.43	50.41	14.00	38.13	12.02	2.72
AntennaPod	9.59	1.89	19.65	9.20	15.43	4.93	4.67
Markor	7.29	1.00	13.77	6.47	9.40	5.00	2.29
F-Droid	9.01	1.49	16.52	7.05	11.42	6.27	2.75
DVAC	6.78	1.14	16.86	6.20	10.20	6.17	0.61
OpenMRS	7.00	0.54	7.66	6.42	8.32	4.93	2.07

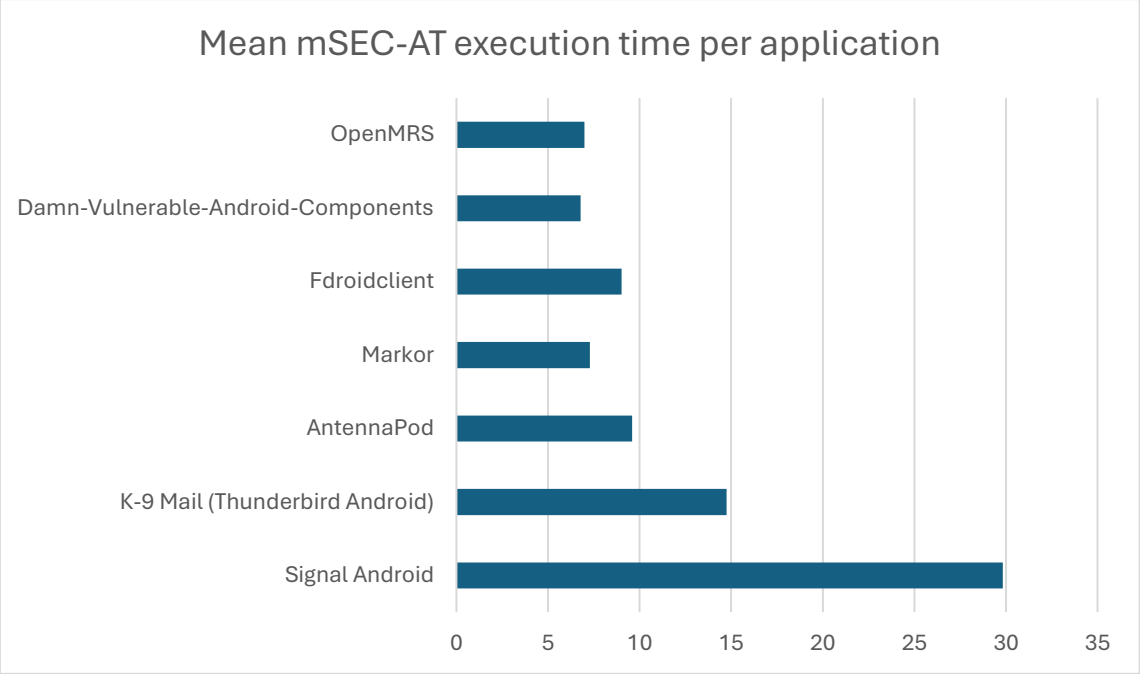

Figure 1. Mean total execution time per application. Signal Android represents the largest subject application and required the longest average runtime.

3. Operation-level execution time

The operation-level measurements indicate that execution time is mainly influenced by source-code analysis and application-processing stages. Across the seven applications, MobSF processing and CodeQL analysis were the longest individual operations on average. By contrast, the LLM-related operations associated with requirement evaluation and report generation were shorter than the main analysis stages in this dataset.

Figure 2 shows the average duration of the main timed operations across the seven subject applications. These operations are reported independently because some of them are executed in parallel or are nested within broader workflow stages; therefore, their durations should not be interpreted as additive components of the total execution time.

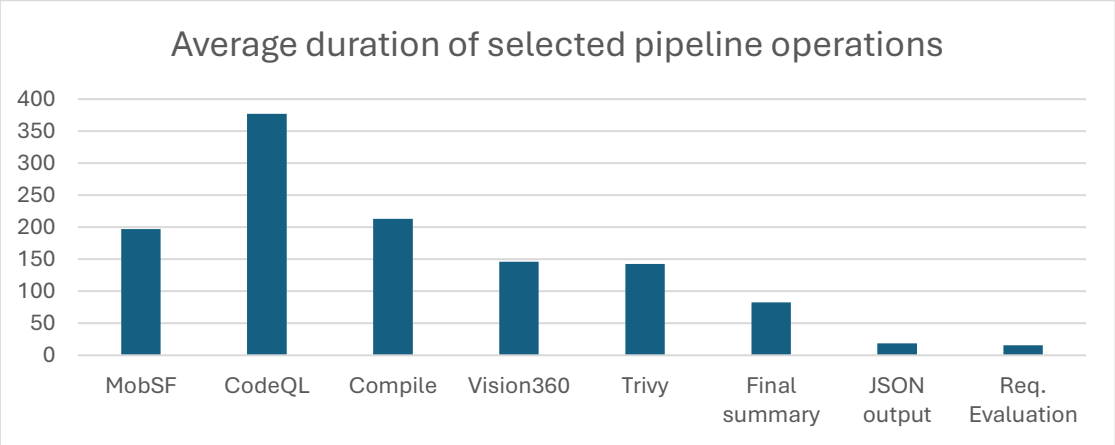

Figure 2. Average duration of selected pipeline operations across the seven subject applications. Operations are timed independently and should not be interpreted as additive percentages of the total runtime.

4. Scalability analysis: repository size and execution time

To explore scalability, the mean total execution time of each application was compared with several size and complexity indicators. Since the sample includes seven applications, the analysis is descriptive and exploratory. The strongest associations are reported in ***Table 7***, and ***Figure 3*** illustrates the relationship between repository size and mean total execution time.

Table 7. Association between application-level indicators and mean total execution time. "Mean": arithmetic mean execution time in minutes; "Repository size (MB)": size of repository in MB.

Application	Mean (min)	Repository size (MB)
Signal	29.83	2360.00
K-9 Mail	14.74	248.00
AntennaPod	9.59	134.00
Markor	7.29	58.20
F-Droid	9.01	79.70
DVAC	6.78	37.30
OpenMRS	7.00	12.10

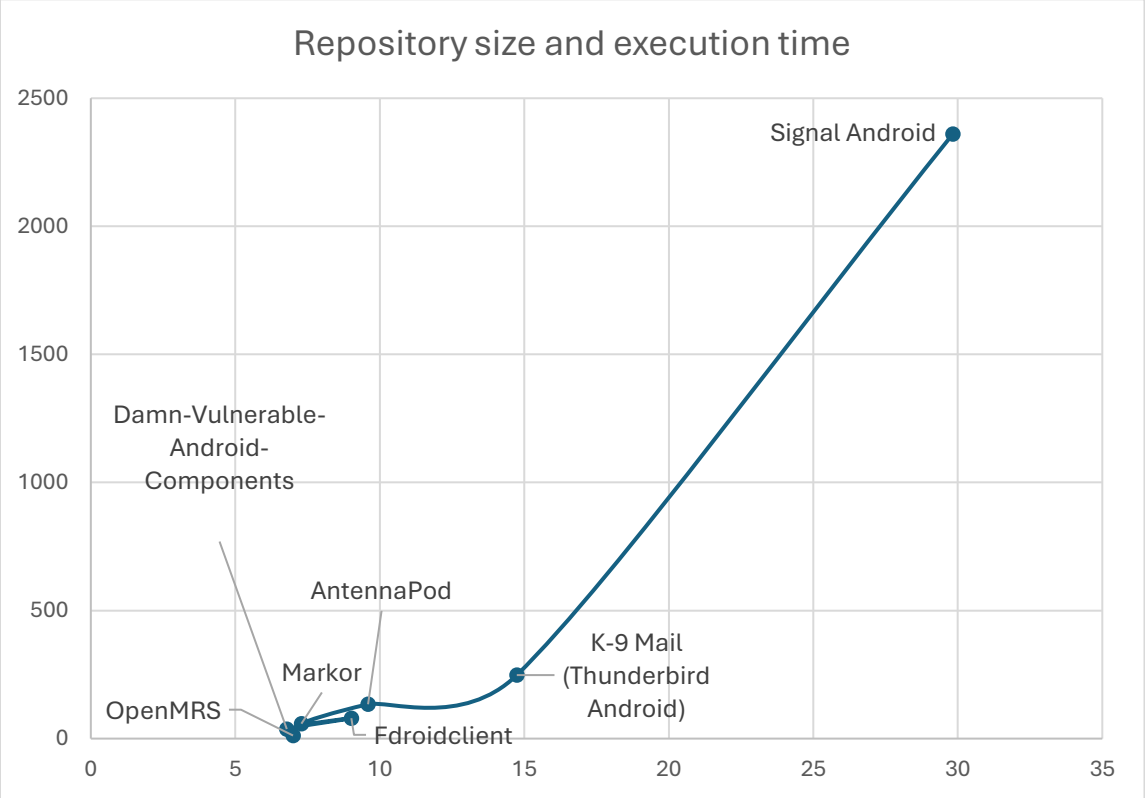

Figure 3. Relationship between repository size and mean total execution time. The dashed line shows the simple repository-size effort-estimation model.

4.1. Runtime distribution by pipeline stage

The phase-level breakdown helps identify which parts of the pipeline contribute most to the elapsed execution time. This is relevant for scalability because different stages depend on different input characteristics: source-code analysis depends on the number and size of source files, MobSF processing depends on the APK and extracted artefacts, and the requirement-evaluation stage depends on the normalised evidence available for mapping against the security catalogue.

Figure 4 shows the relative execution-time distribution by pipeline stage. The parallel analysis block groups the operations executed concurrently during evidence collection, including CodeQL, Trivy, MobSF, and zipped-code processing. Therefore, this block should be interpreted as the elapsed contribution of the parallel stage, not as the sum of all individual tool durations. This representation avoids treating concurrent operations as sequential stages while still showing which part of the pipeline dominates the overall elapsed time.

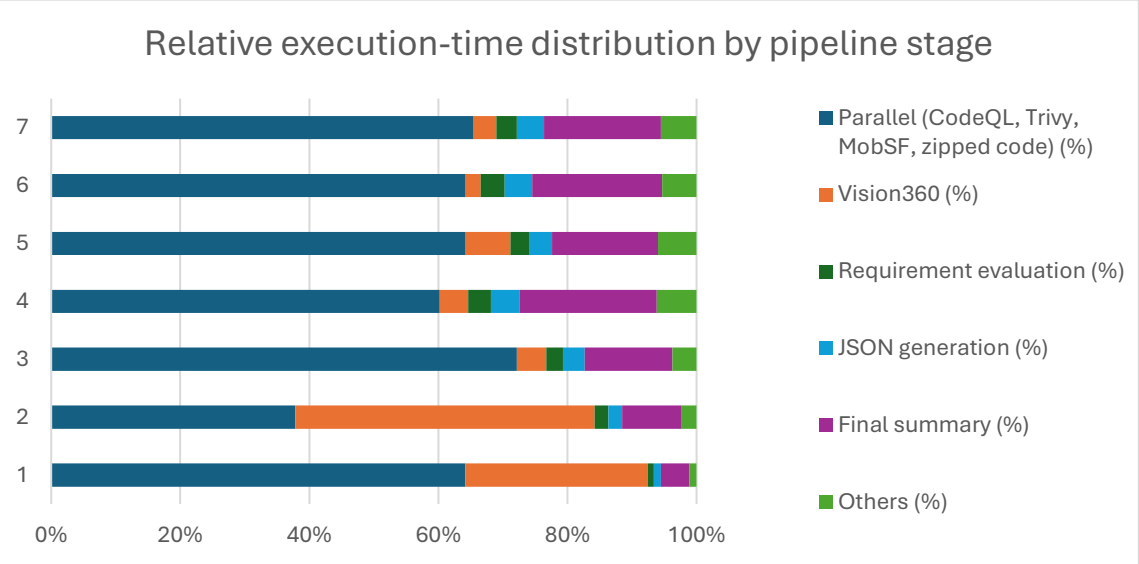

Figure 4. Mean execution-time breakdown by pipeline stage.

5. Association between application characteristics and execution time

Before fitting exploratory estimation models, we analysed the association between mean execution time and application-level characteristics, including repository size, APK size, LOC, scanned files, UFP, Android components, exported components, and findings generated by the integrated tools. Spearman's rank correlation [2] was used to assess monotonic associations without assuming a strictly linear relationship. Given the small number of applications, these results were interpreted as exploratory scalability evidence rather than as definitive statistical generalisation.

The strongest associations were observed for UFP, repository size, total findings, CodeQL findings, Java files, LOC, total scanned files, and Semgrep findings. These results suggest that execution time increases with both application size and the amount of security evidence generated during the analysis. This exploratory analysis informed the selection of simple predictors for the effort-estimation models presented in Section 5.1.

Table 8 summarises the strongest monotonic associations with mean total execution time. Positive Spearman ρ values close to 1 indicate that applications with higher values for a given predictor also tend to have longer execution times. For example, the perfect Spearman correlation observed for UFP indicates that the ranking of applications by functional size matches the ranking by execution time in this dataset. These results should be interpreted as exploratory scalability evidence, since the analysis is based on seven applications and several predictors are interrelated.

Table 8. Strongest monotonic associations with mean total execution time.

Predictor	Spearman rho	Spearman p-value
UFP	1.000	<0.001
Repository Size (MB)	0.964	<0.001
Total Findings	0.929	0.003
CodeQL Findings	0.964	<0.001
Java Files Scanned	0.964	<0.001
Total Files Scanned	0.929	0.003
LOC	0.929	0.003
Semgrep Findings	0.929	0.003
MobSF Risk Findings	0.857	0.014
Android Components	0.371	0.410

Note. Correlations were computed using the mean total execution time per application (n = 7). Spearman's ρ measures monotonic association based on ranks. The results should not be interpreted as causal effects of individual variables, since several predictors are interrelated. Therefore, the correlation analysis should be understood as an exploratory characterization of scalability trends.

5.1. Exploratory execution-time estimation models

Based on the observed positive associations between mean total execution time and application-level indicators such as repository size, LOC, scanned files, UFP, and collected security findings, we fitted exploratory models to estimate total execution time. The purpose of these models is practical rather than explanatory: they provide a preliminary way to approximate the expected execution time of mSEC-AT from metrics that are available either before execution or during preprocessing. However, they should not be interpreted as general predictive laws until they have been validated with a larger and more diverse set of Android applications.

Two complementary models were considered. The first uses only repository size and is intended as a practical early-estimation model, since this metric is available before running the pipeline. The second combines LOC and the number of scanned files, providing a better fit in the evaluated dataset but requiring source-code metrics obtained during preprocessing. Model performance was summarised using the in-sample R^2 and the leave-one-out cross-validation mean absolute error (LOOCV MAE), following the use of cross-validation for assessing statistical prediction performance [3]. ***Table 9*** summarises both models, including their formulae, in-sample R^2, leave-one-out cross-validation error, and intended use.

Table 9. Exploratory models for estimating total execution time.

Model	Predictors	Formula	R^2	LOOCV MAE (min)
Practical repository-size model	Repository size (MB)	Estimated time (min) = 2.477 x (Repository Size MB)^0.307	0.956	2.64
Two-feature source-size model	LOC + total scanned files	Estimated time (min) = 6.505 + (0.00000919 x LOC) + (0.000983 x (Total Files Scanned))	0.993	0.87

Note. R^2 measures the proportion of observed variance in execution time explained by the model in the evaluated dataset. LOOCV MAE is the mean absolute error obtained using leave-one-out cross-validation. With seven applications, LOOCV MAE provides a more conservative indication of estimation error than the in-sample R^2.

The repository-size model can be interpreted as an early planning approximation, since it only requires repository size and can therefore be applied before running the complete pipeline. Its R^2 of 0.956 indicates that repository size explains approximately 95.6% of the observed variance in mean execution time within the evaluated dataset, while its LOOCV MAE of 2.64 minutes indicates an average estimation error of approximately 2.64 minutes under leave-one-out cross-validation. The model follows a sublinear relationship, since the exponent of repository size is lower than 1. Therefore, larger repositories are associated with longer execution times, but the increase is not proportional: doubling the repository size would not be expected to double the estimated execution time.

The two-feature source-size model can be interpreted in practical terms. The intercept of 6.505 minutes represents the approximate baseline cost of executing the pipeline under the evaluated environment, even for small repositories. The LOC coefficient indicates that each additional line of code is associated with an increase of approximately 0.00000919 minutes, or about 0.00055 seconds, in the estimated execution time. The scanned-files coefficient indicates that each additional scanned file is associated with an increase of approximately 0.000983 minutes, or about 0.059 seconds. Therefore, within the evaluated dataset, the model suggests that execution time is mainly associated with

source-code volume and the number of artefacts processed by the pipeline. In contrast with the repository-size model, this model provides a closer fit in this sample, but it is less readily available because it requires measuring LOC and the number of scanned files during preprocessing.

Taken together, these models suggest that the execution time of mSEC-AT can be approximated from simple application-level metrics. Nevertheless, the models should be treated as exploratory. The dataset contains only seven applications, and the predictors are partially correlated with each other. Future evaluations with more applications should be used to recalibrate the coefficients, assess generalisation, and determine whether different families of Android applications require different estimation models.

6. Requirement-level output stability and expert-based validation

In addition to execution time, the evaluation examined the possible variation of the structured audit results across repeated executions. This analysis focuses on the requirement-level classifications produced by mSEC-AT, since these structured results constitute the primary auditable output from which the final report is derived.

For each of the seven applications, the dataset contains 469 security requirements and ten classification columns corresponding to the ten executions. For every requirement, the ten classifications were compared to determine whether the same classification was obtained in all executions. Pairwise agreement was also computed across the 45 possible pairs of executions for each application. ***Table 10*** summarises the resulting requirement-level consistency metrics.

Table 10. Requirement-level output consistency across ten executions. "Yes" (compliant), "No" (non-compliant), and "N/A" indicate the final requirement-level classification categories; "Stable requirements": requirements for which the same classification was obtained in all ten executions; "Inconsistent requirements": requirements with at least one differing classification across executions; "Pairwise agreement": average agreement across the 45 possible pairs of executions for each application.

Application	Yes	No	N/A	Stable requirements	Inconsistent requirements	Pairwise agreement
AntennaPod	26	379	64	469 / 469	0	100.00%
DVAC	28	382	59	469 / 469	0	100.00%
F-Droid Client	37	367	65	469 / 469	0	100.00%
K-9 Mail	22	388	59	469 / 469	0	100.00%
Markor	22	388	59	469 / 469	0	100.00%
OpenMRS	31	373	65	469 / 469	0	100.00%
Signal Android	51	358	60	469 / 469	0	100.00%
Total	**217**	**2635**	**431**	**3283 / 3283**	**0**	**100.00%**

The results show complete consistency across the repeated executions. For all seven applications, the 469 requirements obtained the same classification in the ten executions, resulting in 3,283 stable requirement-level decisions, no inconsistent requirements, and 100% pairwise agreement. This indicates that, under the evaluated configuration, the structured requirement-level output of mSEC-AT is repeatable across executions.

This finding should be interpreted as evidence of repeatability rather than classification correctness. The analysis demonstrates that mSEC-AT produced the same structured classifications when executed repeatedly under the same conditions; however, it does not by itself establish that every classification is correct from a security auditing perspective.

As complementary evidence, expert-based validation provides empirical support for the reliability of the pipeline. In the OpenMRS case study, the AI-assisted audit achieved 85.93% agreement with the human-only baseline across 469 requirements, with Cohen's kappa of 0.628, indicating substantial agreement after accounting for chance. Moreover, no false positives of compliance were observed: the AI-assisted audit never classified a

requirement as Yes when the human baseline classified it as No. This suggests that the pipeline provides conservative and auditable assessments within the reviewed evidence scope.

The repeated-execution analysis and the expert-based validation address two complementary aspects of the trustworthiness of mSEC-AT. The first shows that mSEC-AT produces stable requirement-level classifications under controlled repeated executions, while the second provides evidence that these classifications are broadly aligned with expert judgement in the OpenMRS case study. Nevertheless, the expert validation was limited to one case study, and further validation across additional applications would be required to generalise the observed agreement.

7. Threats to validity

Construct validity. Execution time was measured using the instrumentation available in the pipeline. Requirement-level output consistency was measured by comparing the structured Yes/No/N/A classifications across repeated executions. This captures variation in the structured audit decisions, but not the linguistic variability or explanatory quality of the narrative report.

Internal validity. Execution time may be affected by machine load, dependency resolution, scanner initialisation, Android emulator behaviour, and external service latency. To reduce this risk, all executions were performed using the same dedicated execution environment, and the hardware, network, LLM backend, model, and serving host were kept unchanged.

External validity. The study includes seven Android applications selected to provide heterogeneous size and complexity profiles rather than statistical representativeness. The estimation models require validation with additional applications before they can be used as general predictors of mSEC-AT execution time.

Conclusion validity. Correlation and regression results are exploratory due to the small number of subject applications. Similarly, the 100% requirement-level agreement observed across repeated executions should be interpreted as evidence of repeatability under the evaluated configuration, not as proof that every individual classification is semantically correct. Complementary expert-based validation in the OpenMRS case study provides additional support for classification reliability, but further expert validation across more applications would be required to generalise this result.

8. Data availability

The evaluation is based on 70 complete executions of mSEC-AT. The accompanying execution-time spreadsheet contains the raw execution times, derived metrics, correlations, and exploratory estimation models. A second spreadsheet reports the 469 requirement-level classifications obtained across the ten executions of each application and was used to assess requirement-level output consistency. The LLM prompt catalogue used by the pipeline is also available to support reproducibility of the requirement-level assessment and report-generation stages.

https://github.com/investigaciongiis/mSEC-AT/raw/refs/heads/main/resources/evaluation/mSEC-AT_calculations_times.xlsx

https://github.com/investigaciongiis/mSEC-AT/raw/refs/heads/main/resources/evaluation/mSEC-AT_security_audit_requirements.xlsx

https://github.com/investigaciongiis/mSEC-AT/raw/refs/heads/main/resources/prompts/mSEC-AT_prompts.pdf

9. Conclusions

The evaluation shows that mSEC-AT can be executed on Android applications with different size and complexity profiles, with total execution time increasing mainly with functional size, repository size, LOC, scanned files, and collected findings. The requirement-level consistency analysis also showed no observed variation in the Yes-No-N/A classifications across the ten repeated executions of the seven evaluated applications. Overall, the extended evaluation provides evidence of execution cost, scalability trends, and repeatability of the structured audit outputs under the evaluated configuration.

References

[1] Albrecht AJ. Measuring application development productivity. Proceedings of the Joint SHARE/GUIDE/IBM Application Development Symposium. 1979.

[2] Spearman C. The proof and measurement of association between two things. The American Journal of Psychology. 1904;15(1):72-101.

[3] Stone M. Cross-validatory choice and assessment of statistical predictions. Journal of the Royal Statistical Society: Series B. 1974;36(2):111-147.